```
alex.omegapy@MacStudio CTA_Module-4 % python hospital_robot_astar.py

================================================================================
                          HOSPITAL MEDICATION DELIVERY ROBOT
================================================================================

This classroom program uses A* graph search to find a low-cost route for a hospital
medication-delivery robot. The chosen route must avoid the map's walls and account for the
extra cost of busy corridors and sanitation zones.

Each open map square is one possible robot position. The program defines the robot's legal
moves, the cost of entering each square, the destination check, and an estimate that helps
A* search toward the destination.

================================================================================
                          1. UNDERSTAND THE ROUTING PROBLEM
================================================================================

What the robot is trying to do
------------------------------
The medication-delivery robot needs a low-cost route from one hospital location to another.
It can move one square up, right, down, or left, but it cannot pass through walls. Busy
corridors and sanitation zones are open, although entering them costs more to represent
delay and risk.

  Robot position (state s):             current square, written as (row, column)
  Starting position (state s0):         location where the route begins
  Destination (goal state sg):          location the robot must reach
  Allowed moves (actions a):            UP, RIGHT, DOWN, or LEFT
  Blocked move:                         any move that would enter a wall
  Search stops when:                    the robot's position equals the destination (s == sg)

    Column numbers
    0         10        20        30
 0  ################################
 1  #P.............#.............N#
 2  #.###########...#...###########.#
 3  #..............#..............#
 4  #.#####.#################.#####.#
 5  #.....#.......~~~.......#.......#
 6  #####.#.#####.~~~.#####.#.#######
 7  #.....#.....#..~~~#.....#.......#
 8  #.#########.#..!!!#.###########.#
 9  #...........#.....#.............#
10  #.###########.!!!.###########...#
11  #L............!!!............E..#
12  #.###############.#############.#
13  #S............................C#
14  ################################

How to read the map
-------------------
  #                                 wall (the robot cannot enter)
  .                                 normal corridor (costs 1 unit to enter)
  ~                                 busy corridor (costs 3 units to enter)
  !                                 sanitation zone (costs 6 units to enter)
  P                                 Pharmacy (costs 1 unit to enter)
  N                                 Nurses' Station (costs 1 unit to enter)
  L                                 Laboratory (costs 1 unit to enter)
  E                                 Emergency Department (costs 1 unit to enter)
  S                                 Medical Supply Room (costs 1 unit to enter)
  C                                 Robot Charging Station (costs 1 unit to enter)

Press Enter to choose a starting location and destination...

================================================================================
                          2. CHOOSE THE START AND DESTINATION
================================================================================

Choose from these hospital locations
------------------------------------
  1. P: Pharmacy at (row 1, column 1)
  2. N: Nurses' Station at (row 1, column 31)
  3. L: Laboratory at (row 11, column 1)
```

```
  4. E: Emergency Department at (row 11, column 29)
  5. S: Medical Supply Room at (row 13, column 1)
  6. C: Robot Charging Station at (row 13, column 31)

  Choose the starting location (number/code; Enter = P): c
  Choose the destination (number/code; Enter = E): p

  Starting location (s0):              C - (row 13, column 31) - Robot Charging Station
  Destination (sg):                    P - (row 1, column 1) - Pharmacy

Press Enter to learn how A* scores possible routes...

================================================================================================
                            3. SEE HOW A* SCORES POSSIBLE ROUTES
================================================================================================

How A* decides what to check next
---------------------------------
A* keeps a waiting list, called the frontier, of possible routes. Each route ends at a
search node n. A* gives every node a score and checks the node with the lowest score first.
    f(n) = g(n) + h(n)
    n    = one possible route ending at one map position
    g(n) = exact travel cost already paid (cost so far)
    h(n) = optimistic estimate of the travel cost still left
    f(n) = combined A* score; a lower score is checked first

The estimate h(n) uses Manhattan distance: the vertical row difference plus the horizontal
column difference. It assumes every remaining move costs only 1 unit and temporarily ignores
walls and expensive terrain. That makes the estimate optimistic rather than too large.
    h(n) = (|row - goal_row| + |column - goal_column|) * 1

A* may reach the same map position by more than one route. It remembers the cheapest cost-
so-far value g(n) found for that position and ignores another route when it costs the same
or more.

  Estimated cost left at start h(s0):   42.0 cost units
  Estimated cost left at goal h(sg):    0.0 cost units

Press Enter to let A* calculate the route...

A* is searching for the lowest-cost route... done.

================================================================================================
                                  4. FOLLOW THE ROUTE A* FOUND
================================================================================================

Route map
---------
  A = start, G = destination, * = each square on the selected route
    Column numbers
    0         10        20        30
 0  ################################
 1  #G.............#.............N#
 2  #*###########...#...##########.#
 3  #*******........#..............#
 4  #.#####*#################.#####.#
 5  #.....#*******~~~.......#.......#
 6  #####.#.#####*~~~.#####.#.#######
 7  #.....#.....#**~~~#.....#.......#
 8  #.########.#.*!!!#.##########.#
 9  #..........#.****#............#
10  #.##########.!!!*##########...#
11  #L...........!!!***************#
12  #.###############.#############*#
13  #S............................A#
14  ################################

Route directions (read from left to right)
------------------------------------------
  U means move up, R means move right, D means move down, and L means move left. Each arrow
  means then.
  U -> U -> L -> L -> L -> L -> L -> L -> L -> L -> L -> L -> L -> L -> L -> L -> U -> U ->
  L -> L -> L -> U -> U -> L -> U -> U -> L -> L -> L -> L -> L -> L -> U -> U -> L -> L ->
  L -> L -> L -> L -> U -> U
```

## Route details: one row per move

Step 0 is the starting square; every later row is one move. Move cost is what that move adds. Cost so far is g(n), Est. left is h(n), and A* score is f(n) = g(n) + h(n).

| Step | Move | Position | Terrain / location | Move cost | Cost so far | Est. left | A* score |
|---|---|---|---|---|---|---|---|
| 0 | START | (13, 31) | Robot Charging Station | 0.0 | 0.0 | 42.0 | 42.0 |
| 1 | UP | (12, 31) | normal corridor | 1.0 | 1.0 | 41.0 | 42.0 |
| 2 | UP | (11, 31) | normal corridor | 1.0 | 2.0 | 40.0 | 42.0 |
| 3 | LEFT | (11, 30) | normal corridor | 1.0 | 3.0 | 39.0 | 42.0 |
| 4 | LEFT | (11, 29) | Emergency Department | 1.0 | 4.0 | 38.0 | 42.0 |
| 5 | LEFT | (11, 28) | normal corridor | 1.0 | 5.0 | 37.0 | 42.0 |
| 6 | LEFT | (11, 27) | normal corridor | 1.0 | 6.0 | 36.0 | 42.0 |
| 7 | LEFT | (11, 26) | normal corridor | 1.0 | 7.0 | 35.0 | 42.0 |
| 8 | LEFT | (11, 25) | normal corridor | 1.0 | 8.0 | 34.0 | 42.0 |
| 9 | LEFT | (11, 24) | normal corridor | 1.0 | 9.0 | 33.0 | 42.0 |
| 10 | LEFT | (11, 23) | normal corridor | 1.0 | 10.0 | 32.0 | 42.0 |
| 11 | LEFT | (11, 22) | normal corridor | 1.0 | 11.0 | 31.0 | 42.0 |
| 12 | LEFT | (11, 21) | normal corridor | 1.0 | 12.0 | 30.0 | 42.0 |
| 13 | LEFT | (11, 20) | normal corridor | 1.0 | 13.0 | 29.0 | 42.0 |
| 14 | LEFT | (11, 19) | normal corridor | 1.0 | 14.0 | 28.0 | 42.0 |
| 15 | LEFT | (11, 18) | normal corridor | 1.0 | 15.0 | 27.0 | 42.0 |
| 16 | LEFT | (11, 17) | normal corridor | 1.0 | 16.0 | 26.0 | 42.0 |
| 17 | UP | (10, 17) | normal corridor | 1.0 | 17.0 | 25.0 | 42.0 |
| 18 | UP | (9, 17) | normal corridor | 1.0 | 18.0 | 24.0 | 42.0 |
| 19 | LEFT | (9, 16) | normal corridor | 1.0 | 19.0 | 23.0 | 42.0 |
| 20 | LEFT | (9, 15) | normal corridor | 1.0 | 20.0 | 22.0 | 42.0 |
| 21 | LEFT | (9, 14) | normal corridor | 1.0 | 21.0 | 21.0 | 42.0 |
| 22 | UP | (8, 14) | normal corridor | 1.0 | 22.0 | 20.0 | 42.0 |
| 23 | UP | (7, 14) | normal corridor | 1.0 | 23.0 | 19.0 | 42.0 |
| 24 | LEFT | (7, 13) | normal corridor | 1.0 | 24.0 | 18.0 | 42.0 |
| 25 | UP | (6, 13) | normal corridor | 1.0 | 25.0 | 17.0 | 42.0 |
| 26 | UP | (5, 13) | normal corridor | 1.0 | 26.0 | 16.0 | 42.0 |
| 27 | LEFT | (5, 12) | normal corridor | 1.0 | 27.0 | 15.0 | 42.0 |
| 28 | LEFT | (5, 11) | normal corridor | 1.0 | 28.0 | 14.0 | 42.0 |
| 29 | LEFT | (5, 10) | normal corridor | 1.0 | 29.0 | 13.0 | 42.0 |
| 30 | LEFT | (5, 9) | normal corridor | 1.0 | 30.0 | 12.0 | 42.0 |
| 31 | LEFT | (5, 8) | normal corridor | 1.0 | 31.0 | 11.0 | 42.0 |
| 32 | LEFT | (5, 7) | normal corridor | 1.0 | 32.0 | 10.0 | 42.0 |
| 33 | UP | (4, 7) | normal corridor | 1.0 | 33.0 | 9.0 | 42.0 |
| 34 | UP | (3, 7) | normal corridor | 1.0 | 34.0 | 8.0 | 42.0 |
| 35 | LEFT | (3, 6) | normal corridor | 1.0 | 35.0 | 7.0 | 42.0 |
| 36 | LEFT | (3, 5) | normal corridor | 1.0 | 36.0 | 6.0 | 42.0 |
| 37 | LEFT | (3, 4) | normal corridor | 1.0 | 37.0 | 5.0 | 42.0 |
| 38 | LEFT | (3, 3) | normal corridor | 1.0 | 38.0 | 4.0 | 42.0 |
| 39 | LEFT | (3, 2) | normal corridor | 1.0 | 39.0 | 3.0 | 42.0 |
| 40 | LEFT | (3, 1) | normal corridor | 1.0 | 40.0 | 2.0 | 42.0 |
| 41 | UP | (2, 1) | normal corridor | 1.0 | 41.0 | 1.0 | 42.0 |
| 42 | UP | (1, 1) | Pharmacy | 1.0 | 42.0 | 0.0 | 42.0 |

## Route result

| | |
|---|---|
| Starting location: | C – (row 13, column 31) – Robot Charging Station |
| Destination: | P – (row 1, column 1) – Pharmacy |
| Route found: | Yes |
| Number of moves: | 42 |
| Total travel cost (g at goal): | 42.0 cost units |

## How much work A* did

A node records one possible route to a map position. The frontier is A*'s waiting list of nodes that it may examine next.

| | |
|---|---|
| Expanded nodes (examined): | 86 |
| Generated nodes (created): | 112 |
| Unique states (positions found): | 108 |
| Maximum frontier (largest waitlist): | 26 |
| Stale entries (outdated routes): | 0 |
| Reopened states (searched again): | 0 |

These counts describe only this route request, not A*'s worst case. A* can use a lot of memory on a large map because it keeps its waiting list and the best cost found for each

```
position. This hospital map is small, so the stored information remains manageable.

Press Enter to read why A* fits this problem...

================================================================================
                              5. WHY A* IS A GOOD CHOICE
================================================================================

Why A* is a good fit
--------------------
Evaluation function: The robot must consider both distance and terrain cost. Greedy Best-
First Search uses only h(n), so it may choose a short-looking but expensive route. Uniform-
Cost Search uses only g(n), so it has no estimate pointing toward the destination. A*
balances both values with f(n) = g(n) + h(n).

Completeness: On this finite map, A* will find a route whenever one exists. Every legal move
has a positive cost, and remembering the best cost for each position prevents endless
cycling.

Minimum-cost result: The Manhattan estimate is admissible because it never claims the
remaining trip will cost more than it really must. It is also consistent from one move to
the next. With these properties and positive move costs, this A* graph search returns a
minimum-cost route.

Main tradeoff: A*'s advantage is that it reliably combines exact travel cost with useful
goal direction. Its disadvantage is memory use: on a much larger map, its waiting list and
saved best costs could grow very large. The small classroom map makes that tradeoff
reasonable.

Finished: A* found a route to the destination.
alex.omegapy@MacStudio CTA_Module-4 %
```